

Prompt工程大师课

AI 编程实验室 · 精选教程

¥14.9

生成日期: 2026年05月06日

14 篇精选文章

目录

1. 目录

2. 1. 角色扮演模式：给AI一个身份

3. 2. 示例驱动模式：Few-shot Learning

4. 3. 模板填充模式：结构化指令

5. 4. 链式推理模式：让AI先思考再回答

6. 5. 约束限定模式：设置边界和规则

7. 总结

8. 1. 系统提示词的核心结构

9. 2. 行为约束：防止AI “跑偏”

10. 3. 角色扮演：让AI “入戏”

11. 4. 动态内容注入：让提示词 “活” 起来

12. 5. 多轮对话的状态管理

13. 总结

14. 关于本产品

Prompt 工程大师课

系统学习 Prompt 设计，让 AI 输出你想要的结果。

目录

- 1. Prompt设计模式：让AI输出更精准 (2026-05-06)
- 2. 系统提示词设计：定制AI助手行为 (2026-05-06)

声明： 本内容由 AI 编程实验室自动整理生成，仅供参考学习。

=====

第 1 章：Prompt设计模式：让AI输出更精准

Prompt设计模式：让AI输出更精准

每次和AI对话，最让人抓狂的莫过于“答非所问”。其实，问题往往出在提示词（Prompt）的设计上。本文将分享几种实用的Prompt设计模式，帮助开发者用结构化、可复用的方式，让AI输出更符合预期。

1. 角色扮演模式：给AI一个身份

让AI扮演特定角色，能极大改善输出质量。角色提供了上下文框架和目标导向。

基础写法（不推荐）：

帮我写一个Python函数，检查素数。

角色模式写法：

你是一个资深Python开发者，擅长编写高性能算法。
请实现一个`is_prime(n)`函数，要求：

- 时间复杂度尽可能低
- 包含详细注释
- 提供测试用例

效果对比： 后者会输出更专业、更完整的代码，包含边界处理和性能优化。

实际场景： 让AI扮演“代码审查员”审查你的代码，或扮演“面试官”出技术题。

2. 示例驱动模式：Few-shot Learning

AI在零样本下表现不稳定，但给出几个示例后，模式识别能力会大幅提升。

结构： 输入1 → 输出1 → 输入2 → 输出2 → 新输入 → 期待AI推理输出

示例（提取JSON数据）：

请将以下描述转为JSON格式：

```
描述：张三，28岁，后端工程师
输出：{"name": "张三", "age": 28, "role": "后端工程师"}

描述：李四，35岁，产品经理
输出：{"name": "李四", "age": 35, "role": "产品经理"}

描述：王五，42岁，DevOps工程师
输出：
```

最佳实践： 提供2-3个示例即可，过多反而会分散注意力。示例要覆盖边界情况。

3. 模板填充模式：结构化指令

用模板约束输出格式，特别适合生成报告、代码注释或API文档。

不推荐的写法：

```
解释一下React的useEffect。
```

模板模式写法：

```
请按以下模板解释React Hook: useEffect

【功能概述】：（一句话说明用途）
【使用场景】：（列举2-3个典型场景）
【代码示例】：（一个完整的最小示例，包含import）
【注意事项】：（列出依赖数组、清理函数等关键点）
【常见错误】：（列举1-2个易错点及解决方案）
```

优势： 输出结构固定，便于后续解析或直接嵌入文档。开发者可以轻松“复制粘贴”使用。

进阶技巧： 在模板中使用变量占位符，如 `{hook_name}`，实现批量生成。

4. 链式推理模式：让AI先思考再回答

对于复杂问题，直接要求答案容易出错。引导AI分步推理，效果显著提升。

直接提问：

这段代码的时间复杂度是多少？

```
for i in range(n):  
    for j in range(i, n):  
        print(i, j)
```

链式推理写法：

分析以下代码的时间复杂度。请按步骤推理：

1. 外层循环执行次数是？
2. 内层循环执行次数是？
3. 总执行次数公式是？
4. 最终的时间复杂度是？

代码：

```
for i in range(n):  
    for j in range(i, n):  
        print(i, j)
```

效果：AI会先计算具体次数 $(n + (n-1) + \dots + 1 = n(n+1)/2)$ ，然后推导出 $O(n^2)$ 。推理过程减少了“蒙答案”的概率。

适用场景： 算法分析、数学计算、逻辑推理、调试复杂Bug。

5. 约束限定模式：设置边界和规则

AI经常“发散”，用约束条件控制输出范围。

常见约束类型： - **长度约束：** 限制在100字以内 - **格式约束：** 只返回JSON，不要其他文字 - **知识约束：** 只基于以下文档回答 - **否定约束：** 不要提及任何第三方库

示例（只返回JSON）：

分析以下💎💎户反馈的情感倾向。只返回JSON，不要任何解释。

反馈：这个新版本加载速度太慢了，体验很差。

输出：{"sentiment": "negative", "score": 0.2, "keywords": ["慢", "差"]}

反馈：UI设计很漂亮，功能也很实用，但希望增加夜间模式。

输出：

注意： 约束要具体、可验证。模糊的“写短一点”不如“限制在50字以内”。

总结

本文介绍了5种实用的Prompt设计模式：

- 1. **角色扮演模式** - 给AI身份，提升专业度
- 2. **示例驱动模式** - 用Few-shot引导输出格式
- 3. **模板填充模式** - 结构化指令，输出可控
- 4. **链式推理模式** - 分步思考，减少错误
- 5. **约束限定模式** - 设置边界，防止发散

这些模式可以组合使用。比如“角色+模板+约束”三合一，能让AI在指定角色下，按固定模板输出受限内容，精准度大幅提升。

下次写Prompt时，不妨先思考：我需要AI扮演什么角色？输出格式是怎样的？有哪些边界条件？想清楚这些，AI就不再是“玄学工具”，而是你的高效助手。

=====

第 2 章：系统提示词设计：定制AI助手行为

系统提示词设计：定制AI助手行为

想让AI助手按你的规则行事，而不是天马行空？系统提示词就是你的“遥控器”。本文教你如何通过系统提示词，精准控制AI的角色、行为边界和输出格式。

1. 系统提示词的核心结构

系统提示词是放在用户消息之前的“隐性指令”，它定义了AI助手的全局行为。一个标准的系统提示词包含三个层次：

- **身份定义**：你是谁？扮演什么角色？
- **行为规则**：能做什么，不能做什么？
- **输出规范**：回答的格式、长度、风格。

示例：创建一个Python代码审查助手

你是一位资深Python代码审查专家，拥有10年企业级开发经验。

行为规则：

- 仅回答与Python代码相关的问题
- 发现潜在bug时，必须标注严重等级（Critical/Major/Minor）
- 不提供完整的优化代码，只给出改进建议
- 对于非技术问题，回复“请提供相关代码片段”

输出规范：

- 使用Markdown格式，按“问题→风险→建议”结构组织
- 每个风险点用emoji标记（🐛 Bug | ⚡ 性能 | 🎨 风格）
- 建议包含参考文献链接（Python官方文档或PEP）

应用场景：团队使用AI进行代码审查，确保所有建议格式统一、重点突出。

2. 行为约束：防止AI“跑偏”

没有约束的AI就像脱缰的野马。通过明确的否定规则，可以有效限制AI的行为边界。

示例：一个安全的SQL生成助手

你是一个SQL查询生成助手。

规则：

- 只生成SELECT查询，禁止生成INSERT/UPDATE/DELETE
- 所有表名、字段名必须来自用户提供的数据库schema
- 如果用户请求删除数据，回复：“根据安全策略，我无法生成数据修改语句”

- 生成的SQL必须包含LIMIT子句，防止全表扫描
- 对复杂查询，附加执行计划分析说明

实际场景：给非技术人员提供一个安全的数据库查询接口，避免误操作。

3. 角色扮演：让AI“入戏”

通过详细描述背景、语气和知识范围，可以让AI深度扮演特定角色。

示例：一个Socratic教学法的数学导师

你是一位遵循苏格拉底教学法的数学导师，专攻微积分。

性格特征：

- 从不直接给出答案，总是通过提问引导学生思考
- 语气温和但坚持原则，使用“让我们想想...”句式
- 当学生给出错误答案时，用反例引导，而非直接否定

知识边界：

- 只回答大学本科微积分相关的核心问题
- 对于超出范围的问题，回答：“这是一个有趣的问题，让我们先回到基础的极限概念”
- 必须引用定理名称（如“根据夹逼定理...”）

输出格式：

- 每个回答以一个问题结尾
- 使用LaTeX格式展示数学公式

应用场景：在线教育平台为学生提供智能辅导，保持教学一致性。

4. 动态内容注入：让提示词“活”起来

系统提示词可以包含动态变量，每次调用时注入不同内容，实现个性化。

示例：客户服务助手（带上下文注入）

```
# Python代码示例：动态构建系统提示词
def build_system_prompt(user_info, product_catalog):
    return f"""
    你是一家电商公司的客户服务代表。
    当前用户信息：
    - 用户ID: {user_info['id']}
    - 会员等级: {user_info['tier']}
    - 最近订单: {user_info['last_order']}

    产品目录（仅限以下产品）：
    {product_catalog}
    """
```

```
规则：
- 对VIP会员（等级≥3），提供优先处理通道
- 退货问题必须引导至退货政策页面（URL: /return-policy）
- 回答必须引用产品目录中的具体信息
- 如果用户询问目录外产品，回复：“抱歉，该产品不在我们的服务范围内”

输出格式：
- 先确认用户需求（“我理解您需要...”）
- 提供不超过3个解决方案
- 结尾询问“还有其他需要帮助的吗？”
"""
```

实际应用：电商平台根据用户画像和实时库存，生成个性化客服回复。

5. 多轮对话的状态管理

对于复杂任务，系统提示词需要维护对话状态，确保上下文连贯。

示例：一个分步故障诊断助手

```
你是一个IT技术支持专家，负责网络故障诊断。
当前状态：步骤1/5（问题收集阶段）
步骤流程：
1. 收集症状（当前）
2. 检查硬件连接
3. 运行诊断命令
4. 分析日志
5. 提供解决方案

规则：
- 每步只能问1个问题
- 用户回答后自动推进到下一步
- 如果用户提供的信息不完整，重复当前问题（最多3次）
- 步骤完成后，输出诊断报告模板：
...

## 诊断报告
**问题描述**：[用户描述]
**已执行步骤**：[步骤1-4]
**结论**：[最终判断]
**建议操作**：[步骤5的具体方案]
...

- 如果用户在第2步后说“我不确定”，自动跳到第4步并提供简化方案
```

应用场景：企业内部IT支持系统，引导用户自助排查问题。

总结

设计系统提示词的核心要点：

1. **三层结构**：身份定义 + 行为规则 + 输出规范，缺一不可
2. **明确边界**：用否定规则约束AI，防止越界行为
3. **角色深度**：通过语气、知识范围、互动方式让角色更真实
4. **动态注入**：利用变量实现个性化，避免“千人一面”
5. **状态管理**：对多步骤任务，显式定义状态流转

好的系统提示词就像一份精确的API文档——它告诉AI不仅“做什么”，更重要的是“怎么做”和“不做什么”。下次使用AI助手时，试试用这些技巧给它装上“护栏”。

✦ 关于本产品

- 来源：AI 编程实验室
- 文章数：2 篇
- 生成时间：2026-05-06T05:47:06.495Z

如果觉得有帮助，欢迎分享给需要的朋友！

— END —

由 AI 编程实验室自动生成

<https://hugo-xu844.github.io/autopilot-content/>